

Föreläsning 4

Ändrad
22 Sep
01:10

if--satsen, satsinramning { }

En **if**-sats beslutar vilka satser som ska utföras.

Följande program läser in två tal. Och talar sedan om vilket av talen som är störst.

```
1  #include <stdio.h>
2
3  main()
4  {
5      int i, j;
6
7      printf("Mata in två tal: ");
8      scanf("%d%d", &i, &j);
9
10     if (i > j)
11         printf("%d är störst. \n", i);
12     else
13         printf("%d är störst. \n", j);
14 }
```

Här används **if**-satsen till att bestämma vilken av två **printf**-satser som ska utföras. Om villkoret är sant utförs rad 11 annars rad 13.

Utskrift av körningen :

```
1  Mata in två tal: 12 13
2  13 är störst.
```

Fundera över följande: Vad händer om talen är lika? (Det är inte svårt, jag vill bara lura er att repetera programmet en gång till).

Layout: inskjutning, blanktecken, nya rader

Lägg märke till att raderna 11 och 13 är inskjutna. Detta görs för den mänskliga läsarens skull. För att läsaren ska se vilka satser som hör till **if**-delen respektive **else**-delen.

Datorn däremot, han orienterar sig efter semikolonen. Han bryr sig inte alls om nyrader. Och han bryr sig inte om hur många tecken en rad är inskjuten.

Det här betyder att datorn släpper igenom program som är "fult" layoutade, bara de är korrekta. Ni kan ännu inte skilja mellan "ful" och "snygg" layout av C-program. Så ni har bara ett val: härma. Härma mig eller kursboken.

Notera var kursboken och jag lägger semikolonen. Notera hur många blanktecken vi använder, och var vi sätter dem. Exempelvis runt `=` i tilldelningssatsen. Notera hur långt vi skjuter in rader.

Och gör sedan likadant. Gör boken och jag olika -- välj.

Störst av tre tal

Följande program läser in tre heltal, och avgör sedan vilket av dem som är störst.

```

1  #include <stdio.h>
2
3  main()
4  {
5      int i, j, k, maximum;
6
7      printf("Mata in tre heltal: ");
8      scanf("%d%d%d", &i, &j, &k);
9
10     maximum = i;
11
12     if (j > maximum)
13         maximum = j;
14
15     if (k > maximum)
16         maximum = k;
17
18     printf("Det största talet är: %d\n", maximum);
19 }
```

Här ser vi att det inte behöver finnas en `else`-del i `if`-satsen. Om villkoret på rad 12 eller 15 är falskt görs helt enkelt ingenting.

Idén med programmet är följande: variabeln `maximum` innehåller det värde som är störst av de vi hittills undersökt.

Från början (rad 10) lagrar vi `i` i `maximum`. Sedan jämför vi med `j`. Om `j` är större, så ger vi `maximum` ett nytt värde. Sen gör vi samma jämförelse med `k`.

Ett litet jubileum: Detta är första gången vi ger ett värde till en variabel som redan har ett värde: vi ändrar `maximum` flera gånger om det behövs.

Jag påpekar detta ifall det var någon som trodde att en variabels värde var evigt bestämt efter den första tilldelningssatsen. Så är det inte. Då skulle det inte heta *variabel* -- varierbar. Då skulle det heta *konstant* -- oföränderlig.

Man *kan* definiera konstanter i C-program. Vi återkommer till det.

Utskrift av körningen:

```

1  Mata in tre heltal: 10 15 13
2  Det största talet är: 15
```

Rama in satser med { }

Ofta vill vi att flera satser ska utföras om villkoret är sant. Då måste vi rama in satserna med paret `{` och `}` för att datorn ska förstå. Exempel:

```

1  #include <stdio.h>
2
3  main()
4  {
5      int i, j, diff;
6
7      printf("Mata in två tal: ");
8      scanf("%d%d", &i, &j);
9
10     printf("i = %d, j = %d\n", i, j);
11
12     if (i > j) {
```

```

13         printf("i > j\n");
14         printf("|i - j| = %d\n", i - j);
15     } else {
16         printf("i <= j\n");
17         printf("|i - j| = %d\n", j - i);
18     }
19 }

```

Rad 8 läser in värden till *i* och *j*. Värdena skrivs ut på rad 10. Därefter löps antingen raderna 13--14 igenom *eller* raderna 16--17. Det beror på om *i* är större än *j* eller inte.

I bägge fallen skrivs differensen ut. I detta övningsexempel ville jag ha differensen som ett positivt tal. Därav *if*-satsen för kunna placera *i* och *j* på lämpliga sidor av minustecknet.

Det finns ett smartare sätt att räkna ut differensen. Man kan använda funktionen `abs(i-j)`. Den tar absolutbeloppet. [Se här](#).

Fast det jag huvudsakligen ville visa här var alltså användandet av mjuka parenteser -- `{ }`. Dessa ramar in satserna som ska utföras när villkoret är sant respektive falskt.

Notera också layouten. Jag medger att placeringen av `{` och `}` inte är så snygg runt `else`, men det är inte lätt att hitta plats åt dem, de är i vägen. [Bilting/Skansholm](#) har en annan lösning (se exempelvis sid 101 längst ner).

Den mänskliga läsaren tittar på inskjutningen när han avgör vilka satser som hör till *if*- respektive *else*-delen. Datorn tittar bara efter de mjuka parenteserna.

Slutligen en utskrift:

```

1 Mata in två tal: 10 15
2 i = 10, j = 15
3 i <= j
4 |i - j| = 5

```

Nästa lektion

